IT 401W WEB PROGRAMMING AND FRAMEWORKS FINAL PROJECT A Campus Equipment Borrowing and Inventory System			SCORE
STUDENT NO.	**LAST NAME**	**FIRST NAME**	**MIDDLE INITIAL**

COURSE, YEAR, SECTION, and GROUP	DATE OF SUBMISSION (mm/dd/yyyy)	INSTRUCTOR
		DR. AARON PAUL M. DELA ROSA

PART 1 Project Purpose and Learning Targets

Build a Python Django application that lets campus users request equipment while administrators manage inventory, approval, release, return, maintenance, and operational reports. The system must maintain an accurate history of each physical unit and prevent equipment from being issued twice.

Work in groups of 5–6 members. Every member must contribute code, documentation, testing, and an individual presentation segment. The instructor will announce the submission date; project presentation and Q&A occur in Week 17.

CO	Project evidence	Target PO
CO1	Python and Django backend components	PO-g
CO2	User requirements and MTV design with options comparison	PO-d
CO3	Functional application and tests	PO-e
CO4	Secure data handling and justified privacy/access decisions	PO-n
CO5	ORM relationships queries and reusable modules/libraries	PO-g
CO6	Individual contribution documentation and defense	PO-h and PO-j

Exactly two user levels are permitted: BASIC USER and ADMIN. The admin side must be designed and coded by your group using your own Django views, forms, templates, URLs, and ORM operations. Django authentication is allowed; Django admin pages, admin.site.urls, and packages that generate the admin interface are prohibited.

Approximately three quarters of the required development work must focus on the custom admin side and one quarter on the basic-user side. The admin side handles management, approval, operational workflows, reports, and settings; the basic-user side handles account access, browsing, requests, and tracking.

PART 2 Custom Admin Requirements

Provide dashboard, management, operations, reports, settings, and audit pages. Every protected server-side action must enforce the admin role.

A1 Account management

List, search, activate, and deactivate basic-user accounts through custom pages. Show their request history. Deactivation blocks new sign-ins and transactions without deleting existing records. Registration must never permit a user to select the admin role.

A2 Equipment catalog and physical units
Create, view, update, and archive equipment categories and catalog items. Register physical units with unique asset codes, condition, acquisition date, and status. Filter units by category, item, and status. Archive master records instead of deleting records referenced by transactions.

A3 Request review and approval
Display a searchable, paginated queue filtered by status and date. Review item quantities, requested return date, purpose, and current availability. Approve or reject with a recorded decision and reason. Allocate specific available units during approval; an approved request reserves those units.

A4 Equipment release
Convert an approved request into an issued loan. Record the release date, releasing admin, assigned units, and due date. Print a borrowing slip showing the borrower and asset codes. Prevent issuing a rejected, cancelled, already issued, or insufficiently allocated request.

A5 Partial returns and overdue monitoring
Record returns per physical unit, including date and condition. Support partial returns; close the loan only when all issued units are returned. Classify an outstanding loan as overdue when today is later than its due date. Put a damaged returned unit into maintenance rather than immediately making it available.

A6 Maintenance and inventory reconciliation
Open and close maintenance records with problem, action, dates, and admin. Releasing a repaired unit makes it available; retired units stay unavailable. Reject maintenance creation for a currently issued or reserved unit. Show derived available, reserved, issued, maintenance, and retired counts that reconcile to physical-unit totals.

A7 Dashboard and reports
Provide four live cards: available units, pending requests, active loans, and overdue loans. Add a monthly issued-loan chart and an equipment-category usage chart. Provide a date-filtered loan report and overdue report with CSV export. Derive results from ORM queries and stored transactions.

A8 Audit history and operational settings
Record approval, rejection, cancellation, release, return, maintenance, account changes, and catalog changes with actor, timestamp, action, and record reference. Provide a searchable admin-only audit page. Manage maximum loan duration and maximum requested quantity; validate settings and apply them to new requests.

Dashboard values, charts, reports, and status indicators must change when database transactions change. Static demonstration numbers do not satisfy these requirements.

PART 3 Basic User Requirements

U1 Account and profile
Register, sign in, sign out, and edit only personal name/contact fields. Passwords must be handled by Django authentication. Role and activation status are never editable by the user.

U2 Browse equipment
Search active catalog items by name/category and view description, total serviceable units, and currently available quantity. Hide borrower identities, internal notes, audit records, and maintenance details.

U3 Submit a borrowing request
Submit one request containing one to three different catalog items, quantities, purpose, and desired return date. Validate positive quantities and duration limits. Submission creates a Pending request; it does not approve the request or reserve units.

U4 Track and cancel own requests
View only own request/loan history, status, due dates, and released/returned units. Cancel only Pending or Approved requests before issue. Cancellation of an Approved request frees its reserved units. Users cannot approve requests, release equipment, record returns, or change due dates.

Shared sign-in and authentication infrastructure supports both roles; no third role such as staff, approver, technician, or supervisor may be added.

Role and Access Rules
Use one authentication system with role-based redirects to /manage/ for admins and /portal/ for basic users. Users must not self-promote through registration, profile fields, query strings, hidden inputs, or direct POST requests. Only an authorized admin or a documented setup command may create another admin account.

Scope every user record lookup by the signed-in owner. Protect detail pages, updates, cancellations, AJAX endpoints, reports, and exports. Use active-account checks on protected actions so a deactivated user cannot continue submitting transactions through an existing session.

Show explicit empty states, useful form errors, confirmation before irreversible workflow actions, and status labels that do not depend only on color. Provide consistent navigation and responsive layouts for both sides.

PART 4 Database and Workflow Requirements
Implement the following logical entities with Django models and migrations. You may normalize them further while preserving the required relationships, constraints, and two-role scope.

Entity	Minimum data and relationships
User and Profile	Django authentication identity; role limited to BASIC_USER or ADMIN; name/contact; activation status.
Category and CatalogItem	Category; item name/description; active/archive flag; category relation.
EquipmentUnit	Unique asset code; catalog-item relation; condition; acquisition date; inventory status.
BorrowRequest and RequestItem	Owner; purpose; request/due dates; workflow status; unique item per request; positive quantity.
UnitAllocation	Request-item and unit relations; allocation timestamp; released/cancelled tracking. No simultaneous active allocation for a unit.
Loan and ReturnRecord	One issued loan per request; issue actor/date; individual returned unit/date/condition. Prevent duplicate returns.
MaintenanceRecord	Unit; problem/action; opened/closed dates; responsible admin; maintenance status.
AuditLog and SystemSetting	Actor/action/timestamp/reference; configured quantity and duration limits. No user access.

Workflow
Pending → Approved or Rejected or Cancelled. Approved → Issued or Cancelled. Issued → Partially Returned or Returned; Partially Returned → Returned. Overdue is a calculated label for loans with outstanding units after the due date, not a manually selected request state.

Minimum Demonstration Data
At least 3 categories, 6 catalog items, 18 physical units, 1 admin, 8 basic users, 20 requests, 8 issued loans, and 4 maintenance records. Include rejected and cancelled requests, partial returns, damaged returns, and overdue loans. Seed only fictional identities.

Prepare repeatable fixture files or a management command. Development seed data may include historical dates; ordinary user submission validation must still enforce current date/time rules. Do not use real student names, credentials, or sensitive records.

PART 5 Business Rules and Technical Requirements

1. On approval, choose the required number of available physical units. Reserve all requested units atomically or approve nothing. Release reservations when an approved request is cancelled.
2. An equipment unit has exactly one effective inventory status: Available, Reserved, Issued, Maintenance, or Retired. Return and maintenance operations must maintain this invariant.
3. Use database transactions and a suitable locking or constraint strategy for allocation and issue. Demonstrate two competing approvals for the last unit: only one succeeds.
4. Use a unique constraint for catalog item within a request. For a three-item request, validate every row before saving; a failing row must not leave a partial request.
5. Default settings are a maximum of 7 calendar days and 3 units per request item. A requested due date must be after submission and within the configured duration. Reject issue after the agreed due date; revise through a recorded admin action before issue.
6. Compute overdue days as zero before/on the due date; otherwise use the calendar-day difference for each outstanding loan. Payments, fines, procurement, and supplier portals are outside the required scope.

Required Django Implementation

Use Python and Django with Django ORM, migrations, server-rendered templates, template inheritance, URL names, forms or ModelForms, and Django authentication. SQLite is acceptable for demonstration; PostgreSQL or MySQL may be used. Choose a database appropriate to your concurrency strategy and explain its limitations.

Use one base template for each role and reusable components for navigation, forms, tables, and messages. Provide custom list, detail, create, update, and archive operations where the workflow allows. Never allow unrestricted CRUD to bypass transaction rules.

Use ORM relationships, filtering, ordering, pagination, and aggregate/annotate queries. Keep repeated business logic in reusable functions, services, or model methods. At least one report must combine related models and computed totals.

Validate on the server. Protect state-changing forms with CSRF and use POST rather than GET for updates. Hash passwords through Django authentication, escape user content, check object ownership, and keep secrets out of source control.

HTML/CSS, JavaScript, Bootstrap styling, and a chart library are allowed. AJAX is optional. Third-party packages must be declared and justified; they may not generate the required custom admin side.

Provide a requirements file with tested dependency versions, environment-variable guidance, migrations, seed setup, and a README that starts the application on a clean machine. A working local demonstration is sufficient; paid hosting, external integrations, and online deployment are not required.

PART 6 Submission and Presentation

Required Submission Package

Submit one ZIP named IT401W_ProjectA_Section_Group.zip. Include source code, migrations, requirements.txt, README, fixture/seed command, and the documentation/evidence folder. Exclude virtual environments, caches, real credentials, and private data.

Provide safe demonstration credentials for one admin and two basic users through the instructor-designated submission channel. Include setup instructions rather than committing passwords or application secrets to a public repository.

Submit the following project evidence, with references to source code, records, test results, and documents.

User needs: document requirements, compare two suitable system options, and justify the recommendation. Include usability or acceptance testing, resolved issues, an enhancement responding to user feedback or a changed requirement, and maintenance guidance.

System: provide MTV, database, and workflow diagrams; requirements and constraints; working code; tests; limitations; and enhancements. Explain compliance with relevant standards and legal requirements and how ethical issues are addressed.

Technology: justify selected tools; demonstrate Django, ORM, and reusable components; retain version-control, testing, configuration, and documentation records. Evaluate an emerging technology for effectiveness, compatibility, security, and user needs, and recommend whether to adopt it.

Professional judgment: identify duties and stakeholders; analyze relevant privacy, intellectual-property, security, accessibility, bias, conflict-of-interest, and public-welfare issues. Apply codes, laws, and policies; document and defend decisions, safeguards, risks, recommended actions, and accountability.

Team: submit allocations, collaboration records, individual contributions, and peer feedback. Rotate leadership so every member documents goal-setting, task assignment, coordination, and progress monitoring. Each member evaluates individual/team performance and records improvements made after feedback.

Communication: provide technical setup/development documentation and basic-user/admin guides suited to their audiences. Every member presents an identifiable segment, participates in Q&A, and records important feedback and agreed actions.

Week 17 Demonstration and Q&A
Each group will conduct a 12-minute demonstration followed by a 3-minute Q&A. Demonstrate both user levels and the principal workflows, including validation, access restrictions, conflict handling, and reports. Every member must have an identifiable presentation segment and be prepared to answer questions about the submitted project.

Declare reused code, third-party assets, and permitted AI assistance according to the BulSU AI Policy and instructor instructions. Be prepared to explain the submitted work.

Grading Basis
The project will be graded using all four indicators under each targeted BSIT Universal PO Rubric: PO-d, PO-e, PO-g, PO-h, PO-j, and PO-n. Assessment uses the application, supporting evidence, demonstration, and Q&A, with individual evidence for every member.

Submitted to:

DR. AARON PAUL M. DELA ROSA
Assistant Professor III
Head, College Research and Development Unit
Chair, Local Ethics Review Committee
Certified Python, JavaScript, Java, C++, and C Programmer
DICT ICT Proficiency Examination and NC III Web Development Passer
Certified Data Protection Officer
Associate Member, National Research Council of the Philippines
Member, Institute of Electrical and Electronics Engineers